МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод простых итераций.

Студент гр. 4342 Закаблук А.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 5. Функция $f(x) = arctg(1/x) - x^2$.

Изучить метод Ньютона для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Теоретический минимум.

Метод простых итераций решения уравнения $f(x) = 0$ состоит в замене исходного уравнения эквивалентным ему уравнением x=φ(x) и построении последовательности $x_{n+1}=\varphi(x_n)$, сходящейся при $n\to\infty$ к точному решению. Достаточные условия сходимости метода простых итераций формулируются теоремой, приведенной [1,2,7].

Рассмотрим один шаг итерационного процесса. Исходя из найденного на предыдущем шаге значения x_{n-1}, вычисляется $y=\varphi(x_{n-1})$. Если $|y - x_{n-1}| > \varepsilon$, то полагается $x_n=y$ и выполняется очередная итерация. Если же $|y - x_{n-1}| < \varepsilon$, то вычисления заканчиваются и за приближенное значение корня принимается величина $x_n=y$. Погрешность результата вычислений зависит от знака производной $\varphi'(x)$: при $\varphi'(x)>0$ погрешность определения корня составляет $q\varepsilon/1-q$, а при $\varphi'(x)<0$ погрешность не превышает ε. Здесь q- число, такое, что $|\varphi'(x)|\le q<1$ на отрезке [a,b]. Существование числа q является условием сходимости метода в соответствии с отмеченной выше теоремой.

Для применения метода простых итераций определяющее значение имеет выбор функции $\varphi(x)$ в уравнении $x = \varphi(x)$, эквивалентном исходному. Функцию $\varphi(x)$ необходимо подбирать так, чтобы $|\varphi'(x)|\le q<1$. Это обусловливается тем, что если $\varphi'(x)<0$ на отрезке [a,b], то последовательные приближения $x_n=\varphi(x_{n-1})$ будут колебаться около корня c, если же $\varphi'(x)>0$, то последовательные приближения будут сходиться к корню c монотонно. Следует также помнить, что

скорость сходимости последовательности $\{x_n\}$ к корню с функции $f(x)$ тем выше, чем выше число q.

Выполнение работы.

1) Графический способ. С помощью графического веб-калькулятора desmos.com построим график (Рисунок 1) для определения отрезка $[a, b]$ близкого к $f(x) = 0$ на котором выполняется условие $f(a) * f(b) < 0$.

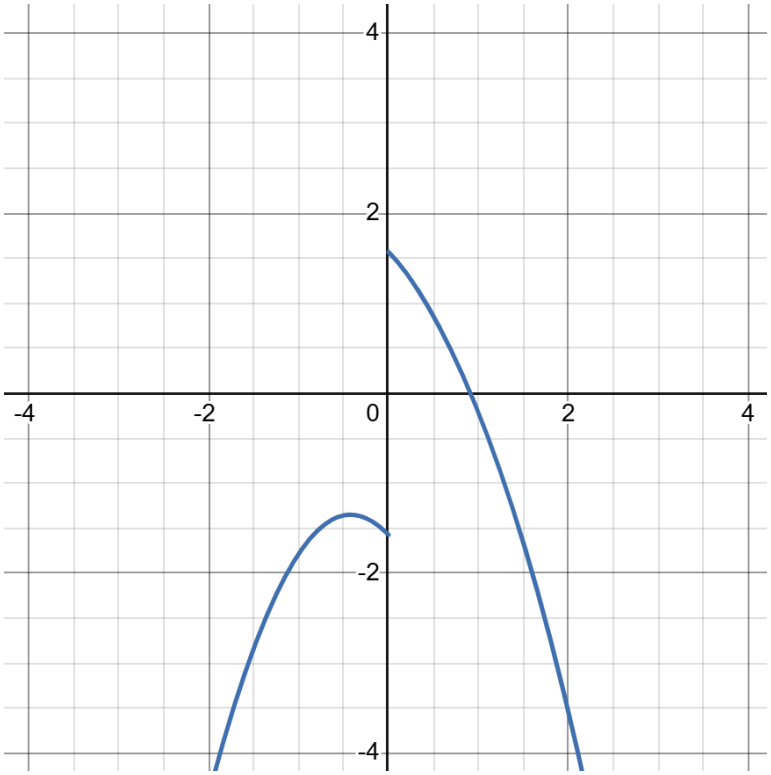

Рисунок 1 - график функции $f(x) = arctg(1/x) - x^2$

На графике видно, что корень находится около значения $x \approx 0.915$. Можно взять отрезок, где a∈(0.1,0.91), а b∈(0.92,2.0). Для простоты и надежности возьмем отрезок [0.5,1.5]. Проверка: f(0.5)≈0.857>0, f(1.5)≈−1.662<0. Условие выполняется.

Аналитический способ (аналитическое отделение корня):

Для доказательства наличия и единственности корня на выбранном отрезке исследуем функцию аналитически с помощью производной. Найдем первую производную функции: $f`(x) = -1/(x^2+1) - 2x$.

Рассмотрим поведение производной на интервале (0,+∞). Очевидно, что при любых x>0 выражения $(-1/(x^2+1))$ и $(-2x)$ строго меньше нуля.

Следовательно, $f'(x) < 0$ при $x \in (0,+\infty)$. Так как производная знакопостоянна и отрицательна, функция $f(x)$ монотонно и строго убывает на всем интервале. Это гарантирует, что функция может пересечь ось Ox не более одного раза (корень единственный).

Проверим выполнение условий теоремы Коши (Больцано-Коши) на границах выбранного отрезка $[a,b]=[0.5,1.5]$:

- $f(0.5)=\mathrm{arctg}(1/0.5)-0.5^2=\mathrm{arctg}(2)-0.25\approx 1.107-0.25=0.857$
- $f(1.5)=\mathrm{arctg}(1/1.5)-1.5^2=\mathrm{arctg}(2/3)-2.25\approx 0.588-2.25=-1.662$

Поскольку функция $f(x)$ непрерывна на отрезке $[0.5,1.5]$ и принимает на его концах значения разных знаков ($f(0.5)>0$, а $f(1.5)<0$, следовательно $f(a)\cdot f(b)<0$), то на этом отрезке существует единственный корень. Аналитическое отделение корня успешно выполнено.

Проверка условий сходимости: Для применения метода простых итераций преобразуем исходное уравнение $f(x)=\mathrm{arctg}(1/x)-x^2=0$ к эквивалентному виду $x=\varphi(x)$. Перенесем x^2 в правую часть и извлечем квадратный корень (берем положительный корень, так как мы ищем решение на положительном отрезке $[0.5,1.5]$): $x^2=\mathrm{arctg}(1/x) \Longrightarrow x=\mathrm{sqrt}(\mathrm{arctg}(1/x))$. Таким образом, наша функция для итераций: $\varphi(x)=\mathrm{sqrt}(\mathrm{arctg}(1/x))$. Проверим условие сходимости $|\varphi'(x)|\le q<1$ на отрезке $[0.5,1.5]$. Найдем производную: $\varphi'(x) = -1/(2(x^2+1)\mathrm{sqrt}(\mathrm{arctg}(1/x)))$.

Оценим модуль производной на концах отрезка:

- При $x=0.5$: $|\varphi'(0.5)|\approx 0.38$
- При $x=1.5$: $|\varphi'(1.5)|\approx 0.20$

Очевидно, что максимальное значение модуля производной на отрезке равно $q\approx 0.38$.Так как $q<1$, достаточное условие сходимости выполнено. Поскольку $\varphi'(x)<0$, последовательность приближений будет сходиться к корню, колеблясь вокруг него.

Выбор начального приближения x_0: Выберем начальное приближение из середины отрезка: x_0=1.0.

Функция extern double F(double x) теперь возвращает значение sqrt(atan(1.0/x)) (т.е. функцию φ(x)), что является особенностью программной реализации метода в данной лабораторной.

2) Функция `extern double F(double x)` теперь возвращает значение `std::atan(1.0, x) - (x * x)` в соответствии с вариантом $f(x) = arctg(1/x) - x^2$. Исходный код в Приложении А.
3) Создана функция, находящаяся в файле main.cpp в Приложении А.
4) Проведены вычисления, построен график (Рисунок 2).

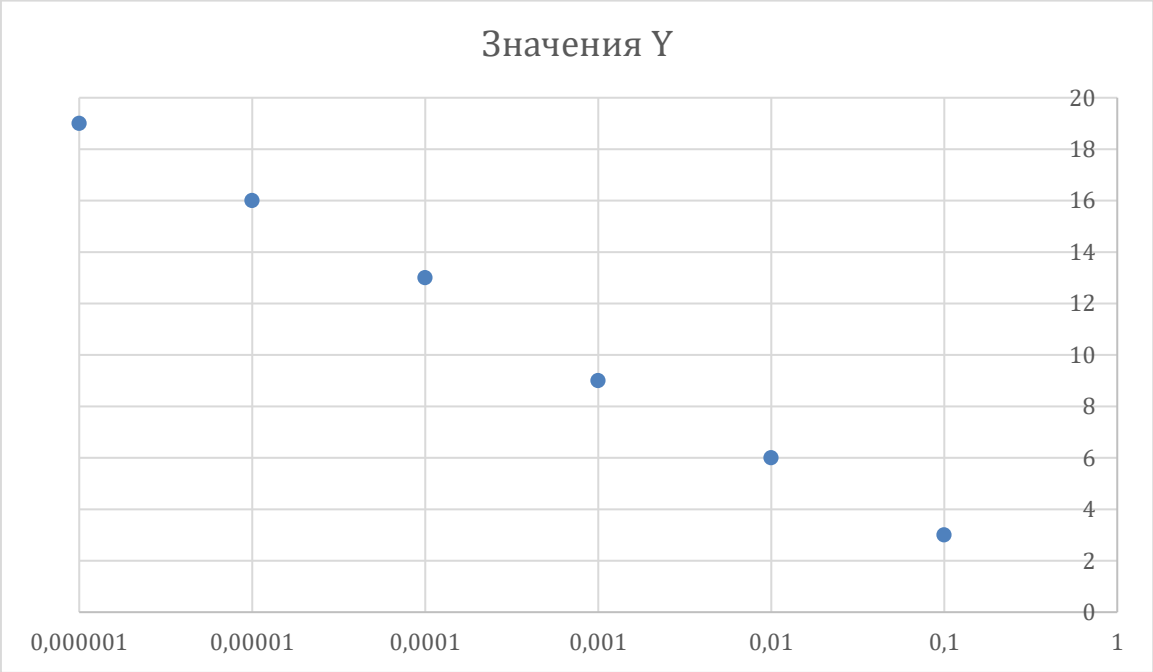

Рисунок 2 - “График” зависимости количества итераций от точности вычислений

Можно заметить, что на Рисунке 2 с ростом числа итераций точность вычислений растет экспоненциально и наоборот, чем меньшую допустимую погрешность (значение ε) мы задаем, тем меньше итераций нужно.

5) Для исследования чувствительности добавлены флаги компиляции ROUND и DELTA. При ROUND=1 DELTA задается автоматически

значением 0.001, если нужно можно задать отдельно. Ниже в Таблице 1 приведены значения при разных дельта.

Таблица 1 - исследование чувствительности метода к ошибкам в исходных данных

Значение DELTA	ε	Значение возвращаемое функцией ITER	Количество итераций необходимых для заданной ε.
10^{-1}	10^{-1}	0.90000	2
	10^{-2}	0.90000	2
	10^{-3}	0.90000	2
	10^{-4}	0.90000	2
	10^{-5}	0.90000	2
	10^{-6}	0.90000	2

Продолжение Таблицы 1

Значение DELTA	ε	Значение возвращаемое функцией ITER	Количество итераций необходимых для заданной ε.
10^{-3}	10^{-1}	0.91400	3
	10^{-2}	0.91500	4
	10^{-3}	0.91500	4
	10^{-4}	0.91500	4
	10^{-5}	0.91500	4
	10^{-6}	0.91500	4
10^{-5}	10^{-1}	0.91428	3
	10^{-2}	0.91521	4
	10^{-3}	0.91521	6
	10^{-4}	0.91521	6
	10^{-5}	0.91521	6

	10^{-6}	0.91521	6
10^{-7}	10^{-1}	0.91428	3
	10^{-2}	0.91521	4
	10^{-3}	0.91522	6
	10^{-4}	0.91521	8
	10^{-5}	0.91521	9
	10^{-6}	0.91521	11

Выводы.

В ходе работы был изучен метод Ньютона. Для уравнения $f(x) = arctg(1/x) - x^2$ корень был успешно найден на отрезке [0.5,1.5] с точностью от 0.1 до 0.000001. Теоретические данные полностью подтвердились экспериментально: метод Ньютона обладает квадратичной скоростью сходимости. Для достижения высочайшей точности (10^{-6}) потребовалось всего 4 итерации, что значительно быстрее методов бисекции (19 итераций) и хорд.

Однако исследование обусловленности показало, что метод Ньютона крайне чувствителен к ошибкам в исходных данных (особенно при вычислении производной). Округление значений функции и производной с низкой точностью ($\Delta=10^{-1}$) приводит к искажению угла наклона касательной, из-за чего алгоритм останавливается на неточном значении и дальнейшее уменьшение ε не улучшает результат.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <cmath>
#include "methods.hpp"

#ifndef X0_START
    #define X0_START 1.0 // Начальное приближение x0
#endif

int main() {
    int iterationsCount;
    std::cout << "Eps\t\t | Root\t\t\t | Iterations\n";
    std::cout << "--------------------------------------------------\n";
    for (double epsilon = 0.1; epsilon >= 0.000001; epsilon /= 10) {
        std::cout << epsilon << "\t\t | "
                  << ITER(X0_START, epsilon, iterationsCount) << "\t\t | "
                  << iterationsCount << '\n';
    }
    return 0;
}
```

Файл methods.cpp

```
#include "methods.hpp"
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#ifndef ROUND_FLAG
    #define ROUND_FLAG 0
#endif
#ifndef DELTA
    #define DELTA 0.001
#endif

#if !ROUND_FLAG
extern double F(double x) {
    // Для метода простых итераций (функция ITER) F(x) должна возвращать
эквивалентную функцию Phi(x), а не саму f(x)=0.
    // Преобразование: x^2 = arctg(1/x) => x = sqrt(arctg(1/x))
    return sqrt(atan(1.0 / x));
}
#ifdef __NEWTON
extern double F1(double x) {
    return (-1.0 / (x * x + 1.0)) - (2.0 * x);
}
#endif
#else
extern double F(double x) {
    return Round(sqrt(atan(1.0 / x)), DELTA);
}
#ifdef __NEWTON
extern double F1(double x) {
    return Round((-1.0 / (x * x + 1.0)) - (2.0 * x), DELTA);
}
#endif
#endif

double HORDA(double Left, double Right, double Eps, int &N)
{
```

```
    double FLeft = F(Left);
    double FRight = F(Right);
    double X, Y;

    if (FLeft * FRight > 0.0) {
        puts("Неверное задание интервала\n");
        exit(1);
    }
    if (Eps <= 0.0) {
        puts("Неверное задание точности\n");
        exit(1);
    }

    N = 0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    do
    {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);
        Y = F(X);
        if (Y == 0.0) return (X);

        if (Y * FLeft < 0.0)
        {
            Right = X;
            FRight = Y;
        }
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    }
    while ( fabs(Y) >= Eps );

    return(X);
}

double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X = (Left + Right) / 2.0;
    double Y;
    if (FLeft * FRight > 0.0) {puts("Неверное задание интервала\n");exit(1);}
    if (Eps <= 0.0) {puts("Неверное задание точности\n");exit(1);}
    N=0;
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;
    while ((Right - Left) >= E)
    {
        X = 0.5 * (Right + Left);
        Y = F(X);
        if (Y == 0.0) return (X);
        if (Y * FLeft < 0.0) Right = X;
        else { Left = X; FLeft = Y; }
        N++;
    };
    return(X);
}
```

```
double Round(double X, double Delta)
{
    if (Delta <= 1E-9) {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0) return (Delta * (long((X / Delta) + 0.5)));
    else         return (Delta * (long((X / Delta) - 0.5)));
}

double ITER(double X0, double Eps, int &N)
{
    if (Eps <= 0.0) {puts("Неверное задание точности\n"); exit (1);}
    // Здесь F() работает как Phi(), возвращая новое X
    double X1 = F(X0);
    double X2 = F(X1);
    N = 2;
    // Особенный критерий остановки, использующий 3 точки (процесс Эйткена)
    while( (X1 - X2)*(X1 - X2) > fabs((2*X1 - X0 - X2)*Eps) )
    {
        X0 = X1;
        X1 = X2;
        X2 = F(X1);
        N++;
    }
    return(X2);
}

#ifdef __NEWTON
double NEWTON (double X, double Eps, int &N)
{
    extern double F1 (double);
    double Y, Y1, DX;
    N = 0;
    do
    {
        Y = F(X);
        if (Y == 0.0) return (X);

        Y1 = F1(X);
        if (Y1 == 0.0) {
            puts("Производная обратилась в ноль\n");
            exit(1);
        }

        DX = Y / Y1;
        X = X - DX;
        N++;
    }
    while (fabs(DX) > Eps);

    return (X);
}
#endif
```

Файл methods.hpp

```
#ifndef METHODS_HPP
#define METHODS_HPP

extern double F(double);

#ifdef __NEWTON
extern double F1(double);
```

```
#endif

double Round(double X, double Delta);
double BISECT(double Left, double Right, double Eps, int &N);
double ITER(double X0, double Eps, int &N);
double HORDA(double Left, double Right, double Eps, int &N);
double NEWTON(double X, double Eps, int &N);

#endif // METHODS_HPP
```